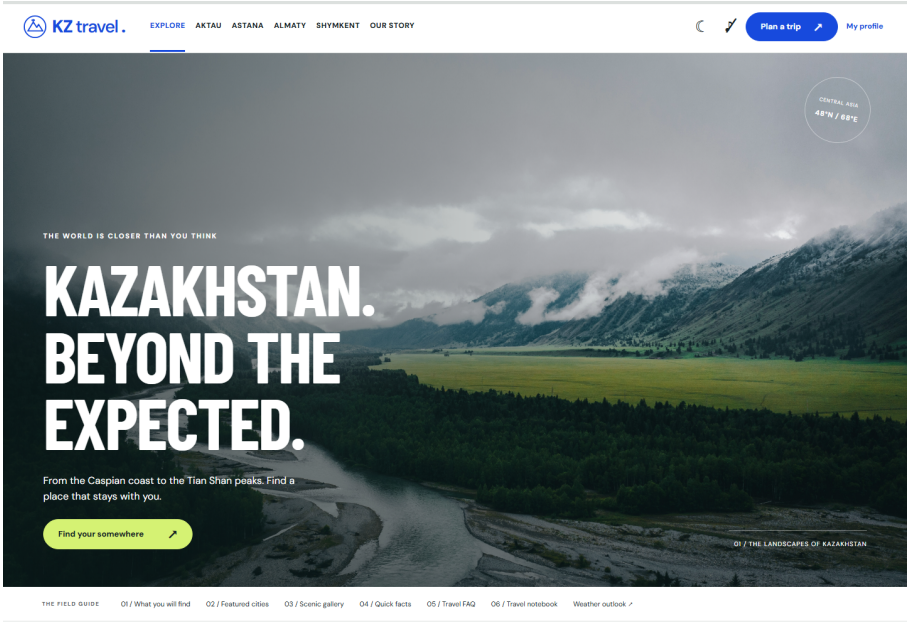
Explore Kazakhstan. Plan your next chapter.



The final KZ Travel homepage uses the original project photographs and shared navigation.

Team	Members and group
KZ Travel	Ramazan Utegen Ziyadinkhan Kudaibergenuly Ruzimuhammad SE-2501

Project type. Multi-page responsive website. Nine connected pages continue the same team topic and previous assignments.

Goal. Help first-time visitors and students compare Kazakhstan destinations, find route ideas, save places and check local weather before a short trip.

Live website: https://ramazanutegen.github.io/assignment1web/

Code repository: https://github.com/RamazanUtegen/assignment1web

Prepared 13 November 2026. Final publication and browser verification are recorded in docs/final/deployment.json.

A travel guide with a clear audience

KZ Travel serves first-time visitors, students and travellers planning short Kazakhstan breaks. It covers Aktau, Astana, Almaty and Shymkent. City content, photographs, quick facts and itineraries support destination choice. The notebook, search, route builder and forecast extend that same purpose.

The project uses HTML, shared CSS, Bootstrap, vanilla JavaScript and jQuery. It is hosted on GitHub Pages without a backend. Original page content and photographs are retained; the final version adds local signup, login, profile and a central JavaScript validator.

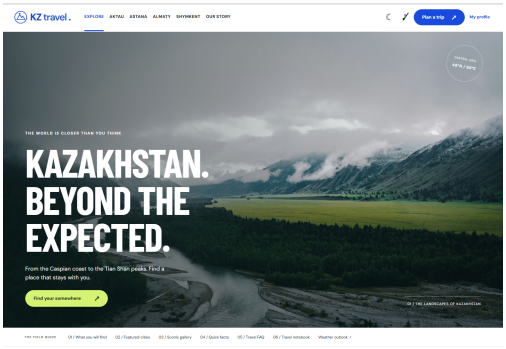
Recorded work division. The existing page credits and team page establish the original contributions below. Shared final modules are documented separately; individual authorship is not inferred from a presentation role.

Member	Original contribution recorded in the project	Presentation role
Ramazan Utegen	Home and Aktau; initial shared visual theme, destination cards, tables and forms.	Introduction, purpose, responsive design and conclusion.
Ziyadinkhan Kudaibergenuly	Astana and Our Story; city itinerary, team profiles, responsive sections and photo journal.	Local account flow and JavaScript validation demonstration.
Ruzimuhammad	Almaty and Shymkent; city information, attraction cards, tables, photographs and enquiries.	Search, saved guides, route builder and API demonstration.

Shared final delivery. Authentication, validation, weather handling, integration and repeatable verification are shared project functionality. Each member should review these modules and rehearse the complete demonstration, regardless of original page ownership.

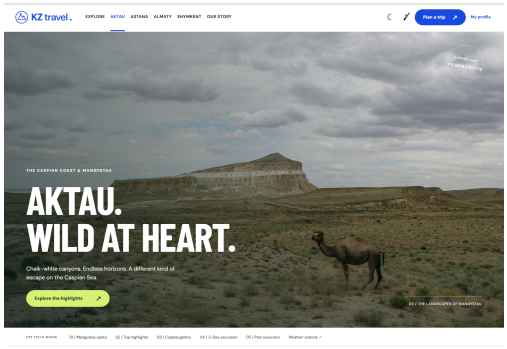
Design decisions. Large destination photographs establish the travel theme. A consistent blue accent identifies actions, lime highlights feedback, and shared spacing and typography connect the pages. CSS variables provide readable day/night palettes without duplicating layouts. New account pages use the same header, footer, imagery and controls.

Six original pages, still complete



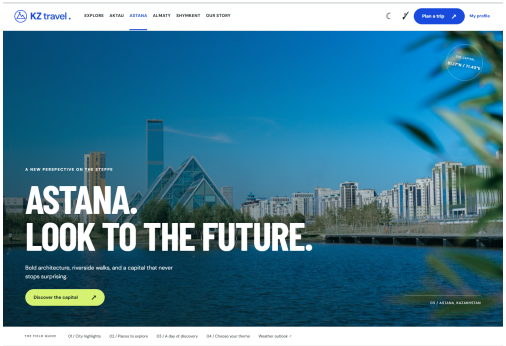
Home

Destination search and filters, FAQ, gallery, weather, three-step planner and links to the notebook.



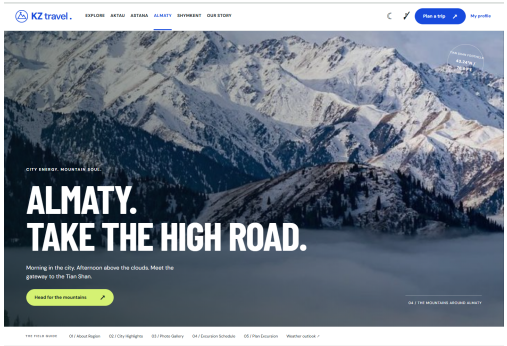
Aktau

Mangystau landscapes, sights, gallery, three-day excursion, local forecast and validated enquiry.



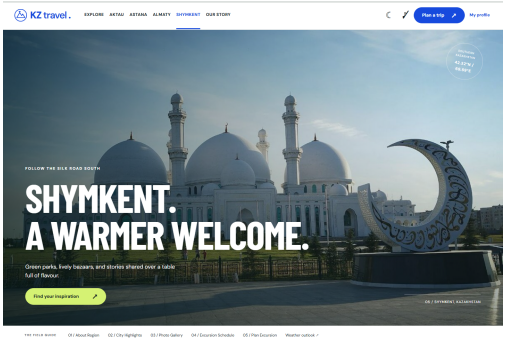
Astana

Capital architecture, places to visit, gallery, itinerary and city forecast.



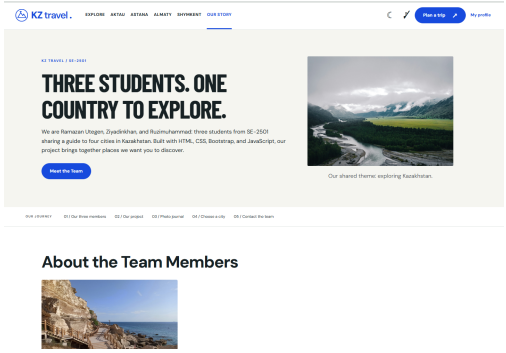
Almaty

City and mountain attractions, photograph gallery, sample itinerary, forecast and enquiry.



Shymkent

Southern culture, parks and food, photo gallery, itinerary, forecast and enquiry.



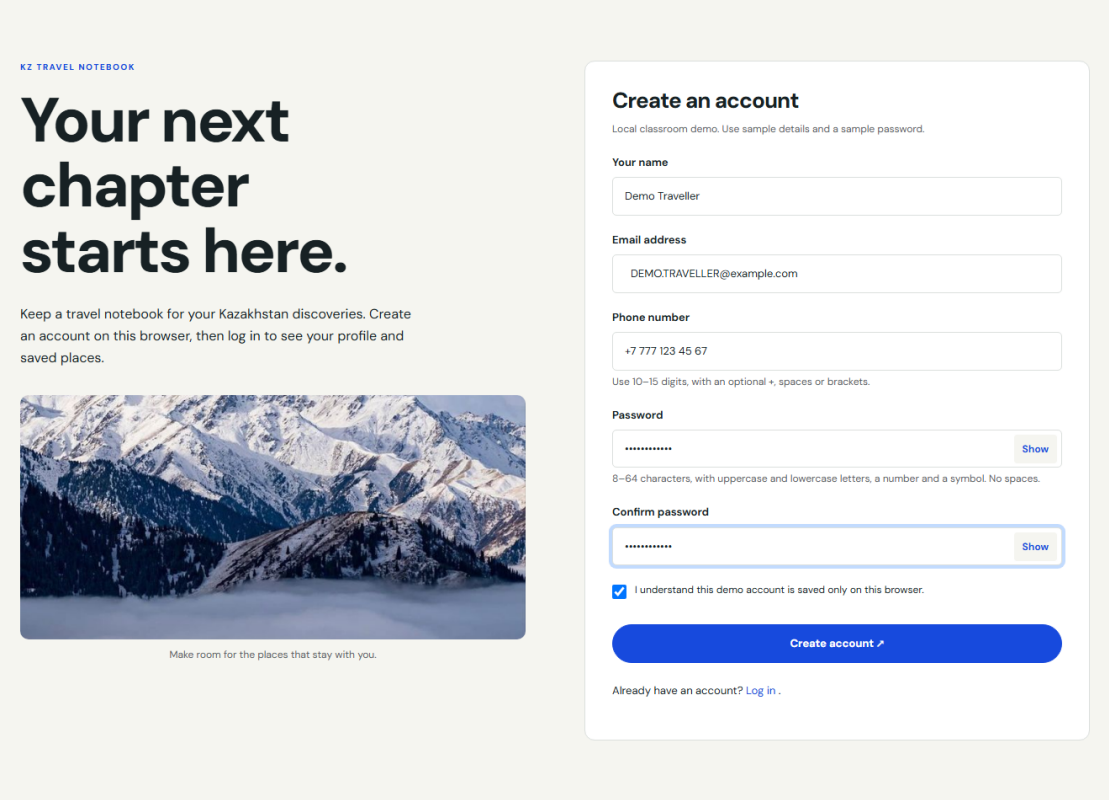
Our Story

Team members, project purpose, photo journal and keyboard-accessible contact dialog.

Sign Up, Log In and Profile add three connected pages, giving nine in total. The public navigation leads to Login, Login links to Signup, and successful authentication opens Profile.

Create a local travel notebook

The Signup page collects name, email, phone, password and confirmation. It validates values before writing account data to localStorage. Duplicate emails are rejected after normalization. A successful registration opens Login with a confirmation message.



Signup uses the shared travel design. The example details are synthetic test data.

The account collection is stored in kz-users-v1. Each account contains contact details and a PBKDF2 password verifier with its own random salt. The plaintext password is never written to localStorage or shown on Profile. Show/Hide buttons change visibility only while entering a password.

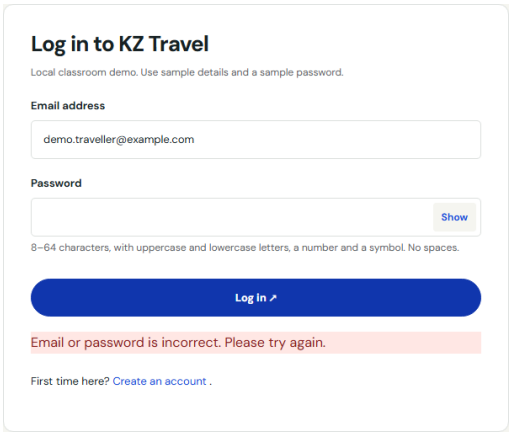
js/auth.js - actual source excerpt; long lines wrap.

```
24     async function passwordHash(password, salt) {
25         if (!window.crypto?.subtle) throw new Error('Open the hosted HTTPS website or use
   localhost to use accounts.');
26         const key = await crypto.subtle.importKey('raw', new TextEncoder().encode(password),
   'PBKDF2', false, ['deriveBits']);
27         const bytes = await crypto.subtle.deriveBits({
28             name: 'PBKDF2', hash: 'SHA-256', iterations: ITERATIONS,
29             salt: Uint8Array.from(salt.match(/../g), value => parseInt(value, 16))
30         }, key, 256);
31         return toHex(new Uint8Array(bytes));
32     }
```

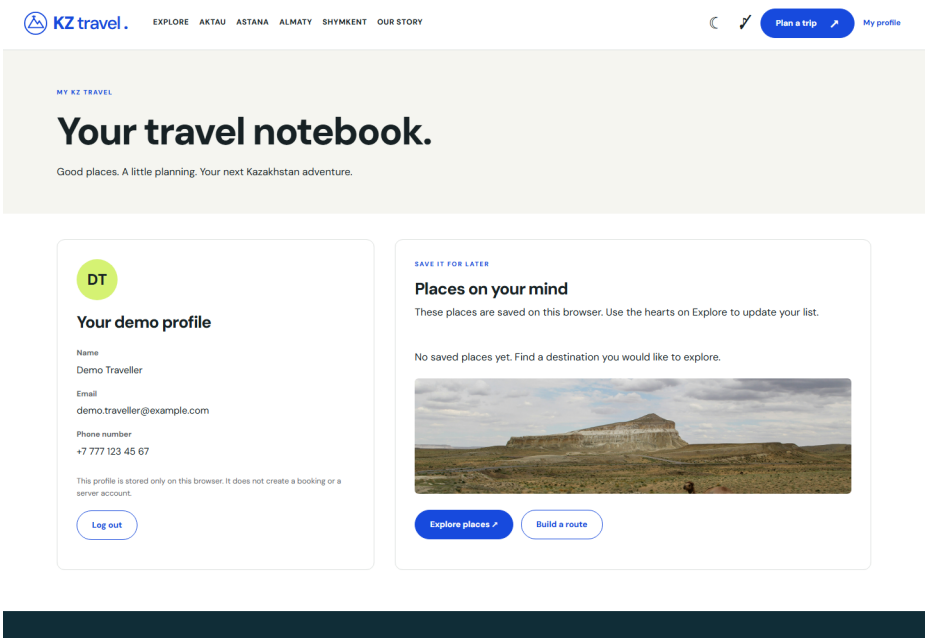
This is a teaching simulation on one browser origin. LocalStorage can be edited in developer tools and does not provide real server authorisation. The interface asks users to use sample details. HTTPS or localhost is recommended for the account flow.

Log in, reload and return

Login checks the normalized email and derives a verifier from the entered password and stored salt. Incorrect credentials produce an inline message. A successful match writes kz-session-v1 and redirects to profile.html.



Incorrect credentials keep the user on Login and show a friendly message.



Profile displays name, email and phone, plus links to cities saved on this browser.

Reloading Profile preserves the local session. Opening it without a valid session redirects to Login. Logout removes the session and returns to Login. A storage-event listener also updates another open profile tab. Saved-city links lead to the existing city guides; empty notebooks include working Explore and Build a route actions.

Saved cities are browser-level preferences and are labelled that way. They are not presented as isolated server-side user data. No booking, message or server account is created.

Validate values before processing

Every form has novalidate. The project does not use required, pattern, minlength, maxlength, min, max or type="email" as its validation mechanism. Explicit data-rules select JavaScript checks; errors are placed next to fields with aria-invalid and aria-describedby.

Field / context	JavaScript rule
Required values	Reject empty and whitespace-only text; required consent must be checked.
Email	Validate address format and reject malformed values; normalize account email case.
Phone	Accept 10-15 digits, optional + and common spaces, brackets and hyphens.
Account password	8-64 characters with uppercase, lowercase, a digit and a symbol; no spaces.
Confirmation	Must match the original password.
Enquiry message	10-2,000 characters. Passwords belong to account forms, not travel enquiries.
Route planner	Valid destination, traveller name of 1-60 characters, and a 1-3 day duration.

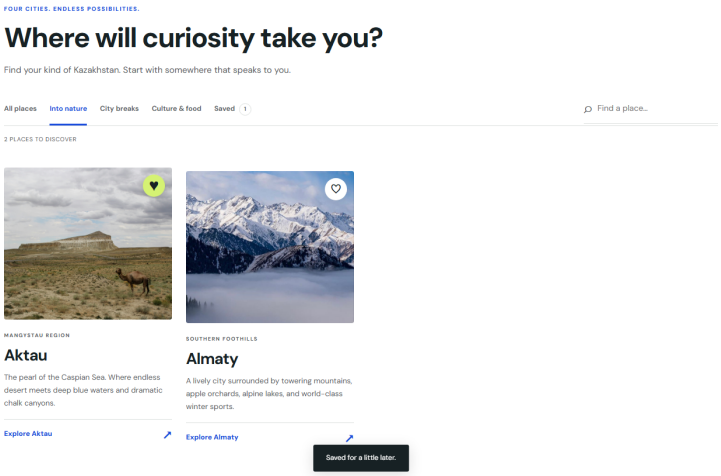
js/validation.js - actual source excerpt; long lines wrap.

```
17         if (rules.includes('phone') && !/^\+?[1-9]\d{9,14}$/.test(value.replace(/[\s()-]/g, ''))) {
18             return 'Use 10–15 digits, optionally starting with +. For example: +7 777 123 45 67.';
19         }
20         if (rules.includes('password') && (raw.length < 8 || raw.length > 64 ||
21             !/[a-z]/.test(raw) || !/[A-Z]/.test(raw) || !/\d/.test(raw) || !/[^A-Za-z0-9\s]/.test(raw) || /\s/.test(raw))) {
22             return 'Use 8–64 characters with an uppercase letter, lowercase letter, number and symbol, without spaces.';
23         }
24         if (rules.includes('confirm') && raw !== form.elements.namedItem('password').value) return 'Passwords must match.';
25         if (rules.includes('message') && (value.length < 10 || value.length > 2000)) return 'Write a message between 10 and 2,000 characters.';
```

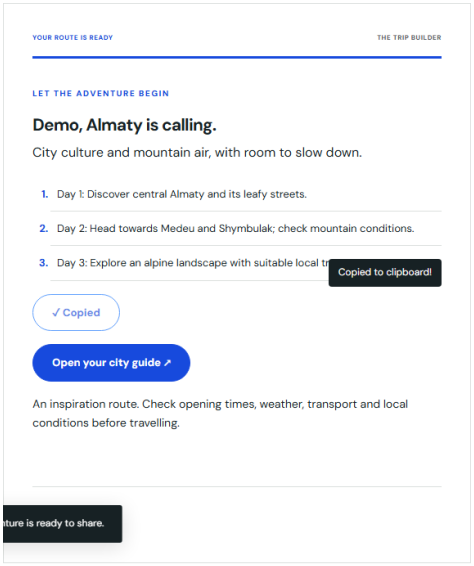
The shared validator is used by both account forms and all four enquiry forms. The planner performs its step-specific checks in JavaScript. Valid enquiries show a spinner, freeze the checked fields and prevent duplicate submission; completion restores controls. The contact dialog keeps keyboard focus within enabled controls.

Find a place and build a route

jQuery listens to search input, combines text with category or Saved filters, updates the results count and offers keyboard/touch autocomplete. Matches are highlighted by building mark elements from text nodes; regular-expression characters are escaped. FAQ search covers both questions and answers.



A category filter and saved-place state update the real destination cards.

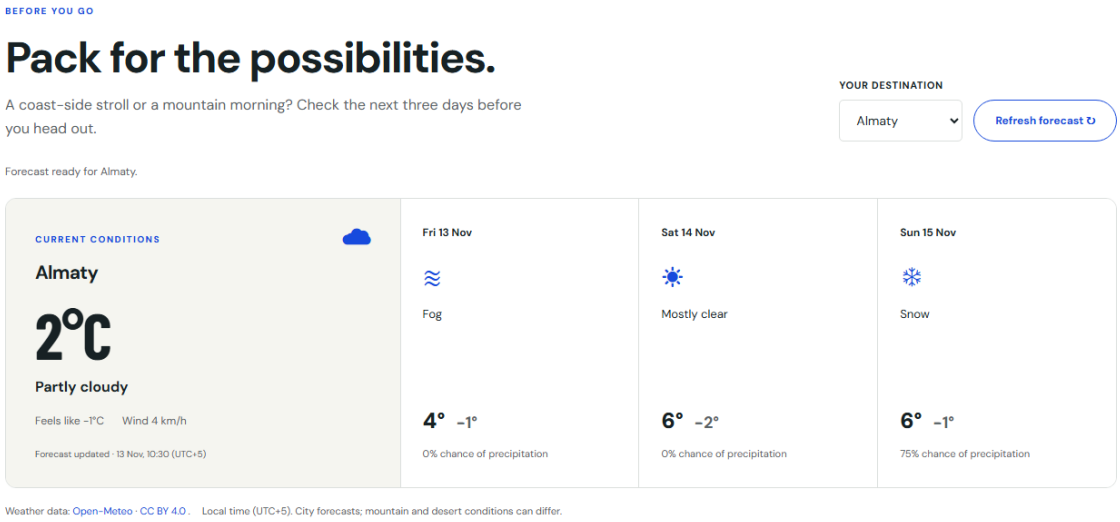


The route builder preserves choices on Back, validates steps and generates a copyable itinerary.

A Set tracks saved city IDs and localStorage persists them. Profile renders the same valid IDs as links. Clipboard confirmation appears only after successful copying. Galleries retain previous/next controls, keyboard handling and thumbnails. Ratings and theme preferences persist locally.

Weather that fits the travel topic

Open-Meteo supplies current conditions and a three-day outlook for the four guide cities. The browser uses fetch and async/await, checks HTTP status, parses JSON and rejects incomplete numeric values. No API key or user geolocation is needed.



Actual Open-Meteo data captured in the browser; weather changes over time.

js/weather.js - actual source excerpt; long lines wrap.

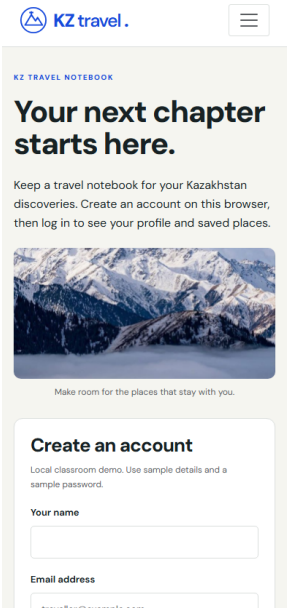
```
152             const response = await fetch(url, { signal: currentController.signal, credentials:
    'omit' });
153             if (!response.ok) throw new Error('Forecast HTTP ' + response.status);
154             const data = await response.json();
155             readForecast(data);
156             if (request !== requestNumber) return; // Never render a late result for another city.
157             saveCache(id, data);
158             render(id, data);
159             panel.dataset.weatherState = 'ready';
```

The forecast loads near the viewport. Users can choose another city or refresh it. Loading and error states are visible. A 10-second timeout restores controls; superseded requests are aborted and a request counter prevents stale results from replacing a newer city.

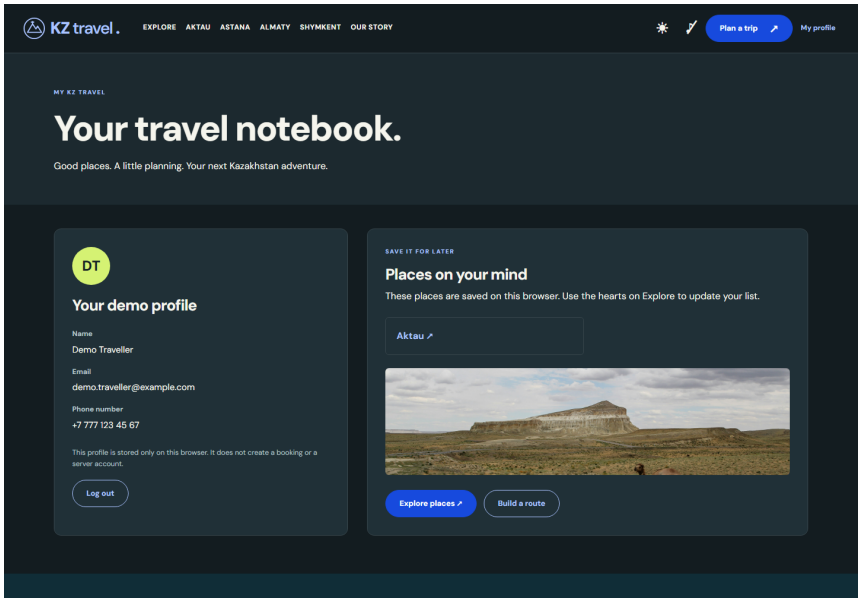
A fresh cache avoids repeat requests for 10 minutes. If an update fails, a clearly dated saved forecast up to 24 hours old may be shown. Expired or corrupt data is ignored. Without usable data the page offers retry and never fabricates a forecast. Source attribution and CC BY 4.0 are shown in the UI.

One interface across screen sizes

Bootstrap, Flexbox and CSS Grid adapt navigation, cards, the weather panel and account pages. The navbar collapses below 1200 px to leave room for the account action on tablets. Text and user details wrap; layouts stack on phones.



Phone signup view.



Profile in the saved night theme.

Browser	Desktop	Tablet	Phone
Chromium	1440 x 1000	1024 x 900	390 x 844
Microsoft Edge	1440 x 1000	1024 x 900	390 x 844

The same theme toggle updates shared CSS variables and saves the selected mode. Tests switch to night on Profile and verify it remains on Login and other pages. Error messages, controls and panels remain readable in both modes. Account details use overflow wrapping so long values do not break columns.

Reduced-motion preference removes unnecessary transitions and counter movement. Skip links, field labels, inline error descriptions, menu Escape, dialog focus management and accessible status messages support keyboard use. Without JavaScript, city content, navigation, photographs and FAQ answers remain available; account pages explain that JavaScript is needed.

Keep feedback useful and code readable

Hover and press effects identify clickable actions. Counters animate once when entering view. Toasts fade after save/copy actions, and duplicate timers are cancelled. Submit spinners communicate processing and prevent repeated clicks. Optional sound starts only after a user action. No button exists solely as decoration.

File / folder	Responsibility
index.html, city HTML, about.html	Travel content, connected navigation, structured sections and images.
signup.html, login.html, profile.html	Account and notebook pages that reuse the shared design.
js/validation.js and js/forms.js	Explicit validation and enquiry completion.
js/auth.js	Account storage, salted verifier, session, profile and logout.
script.js	jQuery search, highlights, notification, clipboard, spinner and lazy images.
js/experience.js	Route planner, theme, saved cities, ratings and Web Audio.
js/weather.js	API, response validation, cache and forecast states.
css/, images/, fonts/, vendor/	Shared presentation, original photos and local dependencies.
scripts/ and docs/final/	Repeatable checks, report/package builders and presentation backups.

Complex sections include comments explaining password verification, stale-request guards and callbacks. The clean submission preserves folder structure and excludes Git history, temporary files and earlier submission archives.

Check the user journeys, then publish

Static verification checks nine connected pages, resources and fragments, unique IDs, labels, jQuery integrity and the prohibition on native validation constraints. Browser journeys run in Chromium and Edge at three sizes.

- Registration: required fields, invalid email/phone, weak and mismatched passwords, duplicate email and no plaintext password persistence.

- Authentication: wrong credentials, successful redirect, profile details, reload persistence, logout and another open profile tab.

- Travel features: combined search/filter, autocomplete, safe FAQ matching, saved guides in Profile, planner, real clipboard text, carousel and ratings.

- Enquiries: inline validation on all city/contact forms, correct phone patterns, frozen values and duplicate-submit rejection.

- Resilience: denied storage restores controls, CDN fallback works, no-JavaScript explanations remain readable, and real weather renders in both themes.

Local browser result. 8 scenario groups passed; JavaScript exceptions and unexpected HTTP failures were not recorded in the main journeys.

Website source: ddce3fb5ed584d5b90a537b8c0d2cbe3edbc5a3f

Published assets: 69 HTML/resources returned HTTP 200 and matched the checkout.

GitHub Pages: successful workflow.

Evidence: docs/final/browser-verification.json, live-browser-verification.json and deployment.json. The source, screenshots and test scripts allow the team to reproduce the demonstrations.

Limitations: accounts are browser-local simulations; enquiries do not send data; external forecasts depend on API availability and may differ from conditions in mountains or deserts. Future work would add a backend, server authentication and personal saved routes.

Three files and a rehearsed demonstration

Every team member submits all three files together in the Final Exam folder:

KZTravel_SE2501_FinalProject.zip

KZTravel_SE2501_Report.pdf

KZTravel_SE2501_Links.txt

The report uses PDF, as explicitly required. The guideline contains a conflicting DOCX example, but its format rule and penalty list reject DOCX. The TXT file lists the live website and repository separately.

Time	Speaker	Focus
0:00-0:30	Ramazan	Team, topic, goal and audience
0:30-1:15	Ramazan	Design, responsive layout and themes
1:15-2:05	Ziyadinkhan	Signup, login and profile
2:05-2:45	Ziyadinkhan	JavaScript validation
2:45-3:35	Ruzimuhammad	Search, saves and route builder
3:35-4:20	Ruzimuhammad	Real weather API
4:20-4:45	Ramazan	Strengths, future work and thanks

The presentation script targets 4:45. Each member speaks or demonstrates. Rehearse with a stopwatch; the timing is a plan, not a claim that rehearsal has already happened. Show the final product rather than editing code live. Screenshots and slide backups are available if connectivity fails.

Deadline supplied by the assignment: 18 November 2026, 23:59. Verify the LMS display and submit by the deadline. The stated cutoff is nine hours later.

References and attribution. Final Project Requirements.pdf and Final Project Presentation Guideline.pdf supplied by the team; official Open-Meteo API documentation; MDN Web Crypto and Local Storage documentation. Bootstrap, jQuery and font licence files remain in the project. Original project photographs are preserved.

Open-Meteo API

Web Crypto PBKDF2

Local Storage